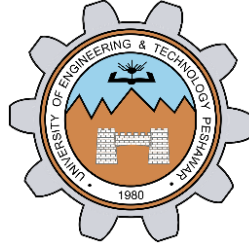


Top down and Bottom up Design Methodologies

LAB # 02



Spring 2024

CSE-308L Digital System Design Lab

Submitted by: **Ali Asghar**

Registration No.: **21PWCSE2059**

Class Section: **C**

“On my honor, as student of University of Engineering and Technology, I have neither given nor received unauthorized assistance on this academic work.”

Submitted to:

Dr. Asif Ali Khan

Date:

24th February 2024

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar

Objectives:

This lab will enable students to:

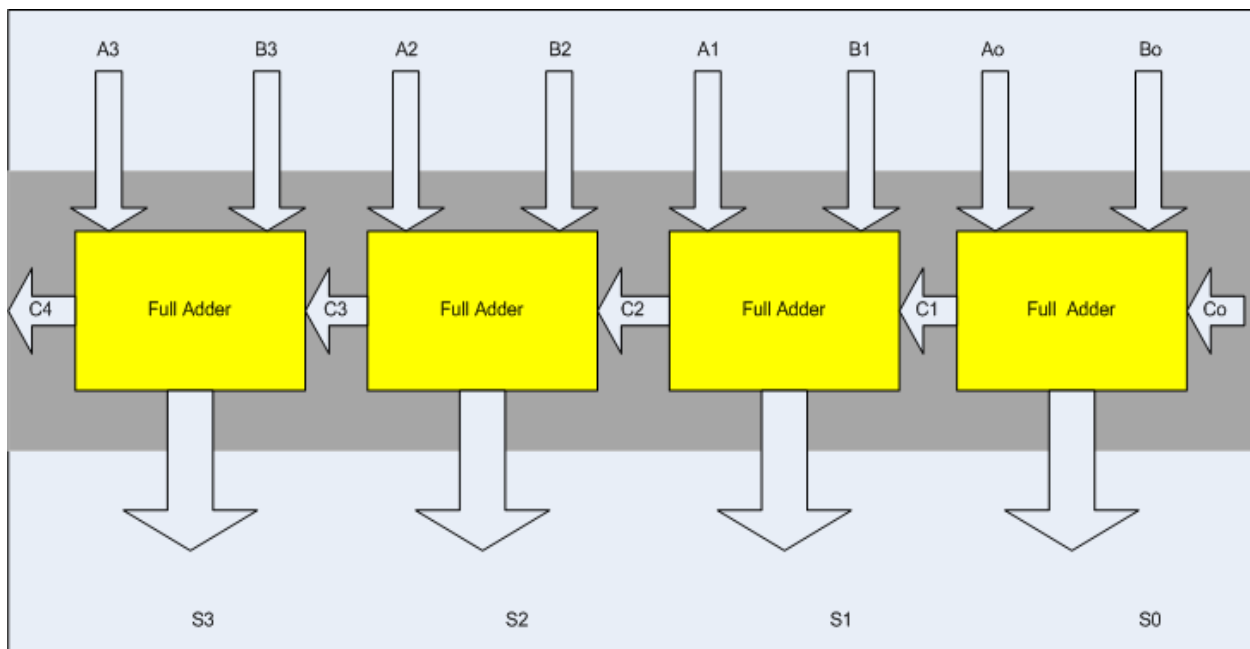
- Learn top down and bottom up design methodologies
- Data flow level modeling

Task 1:

4-bit Ripple Carry Adder

Block Diagram:

Following is the block diagram of a 4 bit Ripple Carry Adder.



Steps for task 1a:

The following steps should be performed while designing a 4 bit RCA adder

ModelSim:

- 1- First implement a Full adder using data gate level modeling.
- 2- Simulate the Full adder with a test bench.
- 3- Instantiate the Full adder four times and connect the circuit as shown.
- 4- Now again write a test bench and simulate the 4 bit RCA

Code:

DUT:

```
1 module carry(A, B, C);
2
3     input A,B;
4     output C;
5
6     and a1(C, A, B);
7
8 endmodule
```

Figure 1.1: Carry Module

```
1 module sum(A, B, S);
2
3     input A,B;
4     output S;
5
6     xor x1(S, A, B);
7
8 endmodule
```

Figure 1.2: Sum Module

```
1 module half_adder(A, B, S, C);
2     input A, B;
3     output S, C;
4     sum s1(A, B, S);
5     carry c1(A, B, C);
6
7 endmodule
```

Figure 1.3: Half Adder Module

```
1 module full_adder(A, B, Cin, S, C);
2     input A, B, Cin;
3     output S, C;
4     wire s1, c1,c2;
5
6     half_adder h1(A, B, s1, c1);
7     half_adder h2(Cin, s1, S, c2);
8     or o1(C, c2, c1);
9 endmodule
```

Figure 1.4: Full Adder Module

```
1 module RCA(A, B, Cin, S, C);
2     input [3:0] A, B;
3     input Cin;
4     output [3:0] S;
5     output C;
6     wire c1, c2, c3, c4;
7
8     full_adder f1(A[0], B[0], Cin, S[0], c1);
9     full_adder f2(A[1], B[1], c1, S[1], c2);
10    full_adder f3(A[2], B[2], c2, S[2], c3);
11    full_adder f4(A[3], B[3], c3, S[3], C);
12 endmodule
```

Figure 1.5: 4-bit Ripple Carry Adder

Test Bench:

```
1  module tb_FA();
2      reg A, B, Cin;
3      wire S, C;
4
5      full_adder fa(A, B, Cin, S, C);
6
7      initial begin
8          $monitor("A = %d B = %d Cin = %d S = %d C = %d", A, B, Cin, S, C);
9
10         {A,B,Cin} = 3'b000;
11         #10 {A,B,Cin} = 3'b001;
12         #10 {A,B,Cin} = 3'b010;
13         #10 {A,B,Cin} = 3'b011;
14         #10 {A,B,Cin} = 3'b100;
15         #10 {A,B,Cin} = 3'b101;
16         #10 {A,B,Cin} = 3'b110;
17         #10 {A,B,Cin} = 3'b111;
18     end
19 endmodule
```

Figure 2.1: Test Bench for Full Adder

```
1  module tbb_RCA();
2      reg [3:0] A, B;
3      reg Cin;
4      wire [3:0] S;
5      wire C;
6
7      RCA rca(A, B, Cin, S, C);
8
9      initial begin
10
11         $monitor("A = %d B = %d Cin = %d S = %d C = %d", A, B, Cin, S, C);
12
13         Cin = 0; A = 4'b0000; B = 4'b0001;
14         #10 Cin = 0; A = 4'b1001; B = 4'b1001;
15
16
17     end
18 endmodule
```

Figure 2.2: Test Bench for 4-bit Ripple Carry Adder

Output:

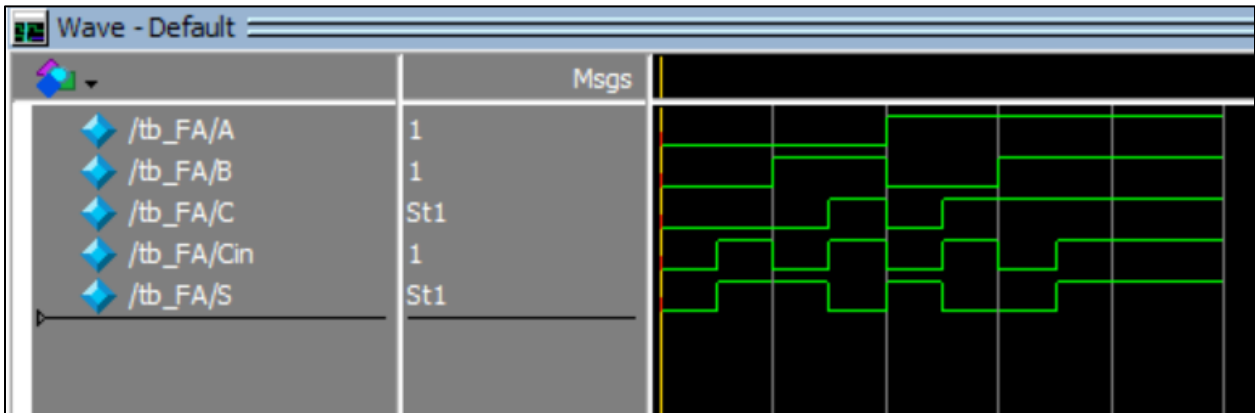


Figure 3.1: Output Waveform of Full Adder

```
VSIM 16> run
```

#	A = 0	B = 0	Cin = 0	S = 0	C = 0
#	A = 0	B = 0	Cin = 1	S = 1	C = 0
#	A = 0	B = 1	Cin = 0	S = 1	C = 0
#	A = 0	B = 1	Cin = 1	S = 0	C = 1
#	A = 1	B = 0	Cin = 0	S = 1	C = 0
#	A = 1	B = 0	Cin = 1	S = 0	C = 1
#	A = 1	B = 1	Cin = 0	S = 0	C = 1
#	A = 1	B = 1	Cin = 1	S = 1	C = 1

Figure 3.2: Truth Table of Full Adder

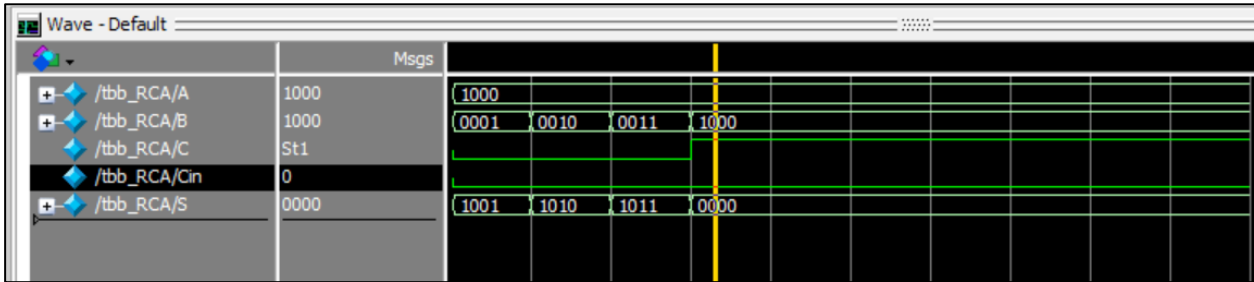


Figure 3.3: Output Waveform of 4-bit Ripple Carry Adder

#	A =	8	B =	1	Cin =	0	S =	9	C =	0
#	A =	8	B =	2	Cin =	0	S =	10	C =	0
#	A =	8	B =	3	Cin =	0	S =	11	C =	0
#	A =	8	B =	8	Cin =	0	S =	0	C =	1

Figure 3.4: Output Table of 4-bit Ripple Carry Adder

Task 1b:

Design the 4 bit full adder using data flow level modeling.

Code:

DUT:

```

module FAD(A, B, Cin, S, C);
    input A, B;
    input Cin;
    output S;
    output C;

    assign S = A^B^Cin;
    assign C = (A & B) | (B & Cin) | (A & Cin);

endmodule

```

Figure 4.1: Full Adder Module Using Data Flow Modelling

```

1  module RCA_D(A, B, Cin, S, C);
2      input [3:0] A, B;
3      input Cin;
4      output [3:0] S;
5      output C;
6      wire c1, c2, c3;
7
8      FA_D f1(A[0], B[0], Cin, S[0], c1);
9      FA_D f2(A[1], B[1], c1, S[1], c2);
0      FA_D f3(A[2], B[2], c2, S[2], c3);
1      FA_D f4(A[3], B[3], c3, S[3], C);
2  endmodule

```

Figure 4.2: 4-bit Ripple Carry Adder Module Using Data Flow Modelling

Test Bench:

```

1  module tb_RCA_D();
2      reg [3:0] A, B;
3      reg Cin = 1'b0;
4      wire [3:0] S;
5      wire C;
6
7      RCA_D rca(A, B, Cin, S, C);
8
9      initial begin
10
11          $monitor("A = %d B = %d Cin = %d S = %d C = %d", A, B, Cin, S, C);
12
13          A = 4'b1000; B = 4'b0001;
14          #10 A = 4'b1000; B = 4'b0010;
15          #10 A = 4'b1000; B = 4'b0011;
16          #10 A = 4'b1000; B = 4'b1000;
17
18      end
19  endmodule
20

```

Figure 4.3: Test Bench for 4-bit Ripple Carry Adder Module Using Data Flow Modelling

Output:



Figure 5.1: Output Waveform of 4-bit Ripple Carry Adder Module Using Data Flow Modelling

```
VSIM 72> run
# A = 8 B = 1 Cin = 0 S = 9 C = 0
# A = 8 B = 2 Cin = 0 S = 10 C = 0
# A = 8 B = 3 Cin = 0 S = 11 C = 0
# A = 8 B = 8 Cin = 0 S = 0 C = 1
```

Figure 5.2: Output Table 4-bit Ripple Carry Adder Module Using Data Flow Modelling